

Tolerância a Faltas

Apresentação Trabalho Prático

Diogo Marques, Francisco Ferreira e Ivan Ribeiro

Implementação do Algoritmo Raft

- Implementado em Rust.
- Ao ser executado, inicializa um nó *Maelstrom* que fica à escuta de mensagens do stdin e escreve para o stdout.

Teste com o *Maelstrom*

```
cargo build --release
java -jar maelstrom.jar test
  -w lin-kv
  --bin ".\target\release\pessi-raft.exe"
  --time-limit 60
  --node-count 3
  --concurrency 10n
  --rate 100
  --nemesis partition
  --nemesis-interval 3
```

Resultado do teste

> Everything looks good!

No entanto, o teste não garante que o algoritmo esteja completamente correto.

O *Maelstrom* só testa um subconjunto limitado de cenários e não garante a cobertura de todos os estados possíveis ou falhas do sistema.

Verificação de Estados via Stateright

O que é o Stateright?

- Ferramenta de model checking para sistemas distribuídos escritos em Rust.
- Permite explorar exhaustivamente os possíveis estados de um sistema e verificar se certas propriedades se mantêm.
- **Tudo a partir de código.**

O *model checking* com Stateright envolve a definição de:

- **Modelo:** Implementação do *Raft*, como modelo;
- **Ambiente:** Rede que não perde, duplica ou reordena mensagens;
- **Propriedades:**
 1. **Segurança de Eleição:** No máximo um líder deve poder ser eleito num mesmo termo.
 2. **Coerência do Log:** Se dois *logs* contêm uma entrada *committed* com o mesmo índice e termo, então as entradas são idênticas.
 3. **Vivacidade do Log:** O *log* não deverá ficar vazio eternamente.
 4. **Vivacidade de Eleições:** Um líder deverá ser eleito inevitavelmente.

Propriedades encontram-se implementadas em cima de código do algoritmo *Raft*.

Exemplo de Propriedade

```
fn election_liveness_property(state: &ActorModelState) -> bool {  
    state  
        .actor_states  
        .iter()  
        .any(|s| s.current_role() == Role::Leader)  
}
```

Exemplo de Uso

```
ActorModel::new((), ())  
  .actors(vec![RaftActor::new(...), RaftActor::new(...), ...])  
  .property(Sometimes, election_liveness_property)  
  .property(Sometimes, log_liveness_property)  
  .property(Always, election_safety_property)  
  .property(Always, log_safety_property)  
  .max_crashes(1)  
  .checker()  
  .target_max_depth(15)  
  ...
```

Demonstração e Mitigações de Faltas

API de Faltas: Injeção de código em pontos críticos da implementação do *Raft*.

- Conseguimos simular anomalias sem alterar o código do algoritmo.
- Avaliação de cada falha via testes unitários (cargo test).
- Verificar se as propriedades são violadas.

Identificação da Falta:

Um nó malicioso pode falsificar ou adulterar mensagens de outros nós.

Impacto:

Extremamente grave. Comprometerá completamente a segurança do *cluster*. Na prática:

- Derrubar eleições legítimas;
- Eleger-se a si próprio, ao manipular votos de outros nós;
- Causar divisões no *cluster*, enviando mensagens contraditórias para diferentes nós;
- Impedir propagação de comandos legítimos.

Demonstração do Impacto:

Demonstração pode ser feita alterando o campo em que é enviado o `node_id` das mensagens.

```
struct VoteRequestMessage {  
    node_id: Id,  
    current_term: TermId,  
    log_length: usize,  
    last_log_term: TermId,  
}
```

No *Stateright*, encontramos um caso que viola a propriedade de **Election Safety**.

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): Legítimo, Id(2): Legítimo

Sequência de Eventos:

1. Timeout(Id(1), ElectionTimeout)
2. Id(1) → VoteRequest { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 } → Id(2)
3. Id(2) → VoteResponse { node_id: Id(2), current_term: 1, vote_granted: true } → Id(1)
4. Timeout(Id(0), ElectionTimeout)
5. Id(0) → VoteResponse { node_id: Id(1), current_term: 1, vote_granted: true } → Id(0)

Estado Inválido Encontrado:

```
Node {  
  id: Id(0),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  current_role: Follower,  
  current_term: 1,  
  ...  
}
```

Propriedades Violadas:

- Segurança da Eleição (Election Safety)

Medidas de Mitigação:

Assinaturas digitais.

Demonstração das Medidas de Mitigação:

Não relacionado com a unidade curricular.

Identificação da Falta:

Um nó malicioso pode votar em dois candidatos diferentes numa eleição no mesmo termo.

Impacto:

Permite que dois nós sejam eleitos como líder, o que quebra a propriedade de **Election Safety**.

Demonstração do Impacto:

Podemos injetar código na receção do `VoteRequest` e responder sempre positivamente ao voto, sem verificar se o nó já votou noutro candidato.

Enviar `VoteResponse` com `vote_granted`: `true` para qualquer `VoteRequest`.

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): Legítimo, Id(2): Legítimo

Sequência de Eventos:

1. Timeout(Id(0), ElectionTimeout)
2. Timeout(Id(2), ElectionTimeout)
3. Id(0) → VoteRequest { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 } → Id(1)
4. Id(0) → VoteRequest { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 } → Id(2)
5. Id(2) → VoteRequest { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 } → Id(0)
6. Id(0) → VoteResponse { node_id: Id(0), current_term: 1, vote_granted: true } → Id(2)
7. Id(1) → VoteResponse { node_id: Id(1), current_term: 1, vote_granted: true } → Id(0)

Estado Inválido Encontrado:

```
Node {  
  id: Id(0),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  current_role: Follower,  
  current_term: 1,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  current_role: Leader,  
  current_term: 1,  
  ...  
}
```

Propriedades Violadas:

- Segurança da Eleição (Election Safety)

Medidas de Mitigação:

Mensagem adicional `ElectedBy` difundida no fim da eleição por todos os nós eleitos no fim da eleição, com a lista de nós que votaram neles.

Se um nó votou em mais de um candidato, este pode ser adicionado a uma *black-list*.

Novo problema: Como garantir que um nó malicioso não possa alterar a lista de votantes?

Demonstração das Medidas de Mitigação:

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): L. com mitigação, Id(2): L. com mitigação

Propriedades a verificar:

- O nó malicioso é adicionado à *black-list*.

O nó malicioso ainda pode fazer com que outro nó se torne líder imediatamente após a eleição (Violação da propriedade de **Election Safety**).

Não há problema porque é invocada uma nova eleição de seguida. Mas não podemos verificar todas as propriedades.

Sequência de Eventos:

1. Timeout(Id(2), ElectionTimeout)
2. Id(2) → VoteRequest { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 } → Id(0)
3. Timeout(Id(1), ElectionTimeout)
4. Id(1) → VoteRequest { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 } → Id(0)
5. Id(2) → VoteRequest { node_id: Id(2), current_term: 1, log_length: 0, last_log_term: 0 } → Id(1)
6. Id(0) → VoteResponse { node_id: Id(0), current_term: 1, vote_granted: true } → Id(1)
7. Id(0) → VoteResponse { node_id: Id(0), current_term: 1, vote_granted: true } → Id(2)
8. Id(1) → Other(ElectedBy { leader: Id(1), term: 1, by: [Id(1), Id(0)] }) → Id(1)
9. Id(2) → Other(ElectedBy { leader: Id(2), term: 1, by: [Id(2), Id(0)] }) → Id(1)

Estado Inválido Encontrado:

```
Node {  
  id: Id(0),  
  blacklist: [],  
  current_role: Follower,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  blacklist: [Id(0)],  
  current_role: Follower,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  blacklist: [],  
  current_role: Leader,  
  ...  
}
```

(ElectedBy ainda não foi propagado para o nó 2)

Propriedades Verificadas:

- Nó malicioso (0) é adicionado à *black-list*.

Identificação da Falta:

Nó malicioso pode ignorar *timer* de eleições e enviar sempre VoteRequest para todos os outros nós, incrementando o termo de eleição.

Impacto:

Não há líder estável, portanto propriedade de **Log Liveness** é violada.

Demonstração do Impacto:

A cada mensagem recebida, o nó malicioso começa uma nova eleição.

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): Malicioso, Id(2): Legítimo

Dois atores devido a que, se fosse apenas um, e os nós 1 e 2 não recebessem as mensagens do nó 0 (*delay* na rede), o sistema conseguiria progredir.

Propriedades Não Verificadas:

- Vivacidade do Log (Log Liveness)

Medidas de Mitigação:

Não há solução concreta. Apenas heurísticas.

Possíveis soluções rondam a ideia de limitar o número de eleições consecutivas ou num certo período de tempo.

Valores concretos terão que ser ajustados para obter um comportamento acertado.

Demonstração das Medidas de Mitigação:

Optámos por uma abordagem mais simples:

- se um nó iniciar eleições em três termos consecutivos, é adicionado à *black-list*.

Ambiente de Teste:

- Atores: Id(0): **Malicioso**, Id(1): **L. com mitigação**, Id(2): **L. com mitigação**
- Propriedades a verificar:
 1. Nó malicioso é adicionado à *black-list*.
 2. Segurança da Eleição (Election Safety)
 3. Vivacidade do Log (Log Liveness)
 4. Vivacidade de Eleições (Election Liveness)
 5. Coerência do Log (Log Safety)

Sequência de Eventos¹:

1. Timeout(Id(0), ElectionTimeout)
2. Id(0) → VoteRequest { node_id: Id(0), current_term: 1, log_length: 0, last_log_term: 0 } → Id(2)
3. Id(2) → VoteResponse { node_id: Id(2), current_term: 1, vote_granted: true } → Id(0)
4. Id(0) → VoteRequest { node_id: Id(0), current_term: 2, log_length: 0, last_log_term: 0 } → Id(2)
5. Id(2) → VoteResponse { node_id: Id(2), current_term: 2, vote_granted: true } → Id(0)
6. Id(0) → VoteRequest { node_id: Id(0), current_term: 3, log_length: 0, last_log_term: 0 } → Id(2)

¹que levaram ao nó malicioso ser adicionado à *black-list*.

Estado Válido Encontrado²:

```
Node {  
  id: Id(0),  
  blacklist: [],  
  current_role: Candidate,  
  ...  
}
```

```
Node {  
  id: Id(1),  
  blacklist: [],  
  current_role: Follower,  
  ...  
}
```

```
Node {  
  id: Id(2),  
  blacklist: [Id(0)],  
  current_role: Follower,  
  ...  
}
```

Propriedades Verificadas:

- Nó malicioso é **adicionado à *black-list***;
- Segurança da Eleição (Election Safety);
- Vivacidade de Eleições (Election Liveness);
- Vivacidade do Log (Log Liveness);
- Coerência do Log (Log Safety);

²Neste estado, só o nó 2 recebeu os pedidos do nó malicioso.

Identificação da Falta:

Um nó malicioso pode enviar, no mesmo índice e termo, duas versões diferentes da entrada do log para nós diferentes.

Impacto:

Se dois conjuntos de nós aplicarem entradas diferentes no mesmo índice e termo, a propriedade de **Coerência do Log** é violada.

Demonstração do Impacto:

Podemos injetar no código de ao receber um LogRequest, incrementar todos os valores das entradas do log do nó malicioso. Fazendo com que se simule que este nó tenha recebido um nó diferente dos outros.³

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): Legítimo, Id(2): Legítimo

³Podia haver uma demonstração mais clara. Limitado pela “API de Faltas”, que não permite injeção de código no envio de mensagens. Futura melhoria.

Sequência de Eventos:

1. Timeout(Id(1), ElectionTimeout)
2. Id(1) $\rightarrow$ VoteRequest { node_id: Id(1), current_term: 1, log_length: 0, last_log_term: 0 } $\rightarrow$ Id(0)
3. Id(0) $\rightarrow$ VoteResponse { node_id: Id(0), current_term: 1, vote_granted: true } $\rightarrow$ Id(1)
4. Id(1) $\rightarrow$ LogRequest { leader_id: Id(1), current_term: 1, prefix_length: 0, prefix_term: 0, commit_length: 0, suffix: [] } $\rightarrow$ Id(0)
5. Id(0) $\rightarrow$ LogResponse { node_id: Id(0), current_term: 1, ack: 0, success: true } $\rightarrow$ Id(1)
6. Id(1) $\rightarrow$ ClientRequest(Write { key: 1, value: 42, msg_id: 1 }) $\rightarrow$ Id(1)
7. Id(1) $\rightarrow$ LogRequest { leader_id: Id(1), current_term: 1, prefix_length: 0, prefix_term: 0, commit_length: 0, suffix: [LogEntry { term: 1, from: Id(1), request: Write { key: 1, value: 42, msg_id: 1 } }] } $\rightarrow$ Id(0)
8. Id(0) $\rightarrow$ LogResponse { node_id: Id(0), current_term: 1, ack: 1, success: true } $\rightarrow$ Id(1)

Medidas de Mitigação:

Ao receber um LogRequest, enviar para todos os outros nós o *hash* do *log* atual até ao *commit_length*. Caso divergir, declarar o líder como *black-list* e começar nova eleição.

No entanto há problemas acrescidos:

- Custoso em termos de mensagens;
- Não haveria forma de reestabelecer o *log* original;
- Nó malicioso podia fabricar *hashes* erradas de modo a banir nós legítimos.

Decidimos não implementar uma solução para este problema.

Identificação da Falta:

Um nó líder malicioso pode incrementar o `commit_length` sem ter recebido confirmação da maioria dos nós.

Impacto:

- Os *logs* podem não estar replicados numa maioria de nós, o que pode levar a inconsistências no sistema.
- Quebra princípio de **Log Safety**, pois o sistema considera *logs* como *committed* mesmo que não tenham sido aceites pela maioria dos nós.

Demonstração do Impacto:

Podemos criar um nó malicioso que, quando é líder, incrementa o `commit_length` para o tamanho do *log* depois de qualquer mensagem ter sido enviada.

Ambiente de Teste:

- Atores: Id(0): Malicioso, Id(1): Legítimo, Id(2): Legítimo, Id(3): Legítimo, Id(4): Legítimo
- Número de *crashes* possíveis: 5
- Mensagens possíveis:
 - `ClientRequest::Write { key: 1, value: 42, msg_id: 1 }`
 - `ClientRequest::Write { key: 2, value: 43, msg_id: 2 }`

Apesar de aumentarmos as variáveis possíveis no ambiente de teste, o *Stateright* não encontrou qualquer violação das propriedades.

Porquê?

- Limitações da exploração do espaço de estados?
- Impossibilidade de simular partições de rede?
- Inexistência do problema?

Não conseguimos encontrar o motivo a tempo útil.

Medidas de Mitigação:

Obrigar ao líder a acumular assinaturas digitais que são produzidas quando os nós enviam pacotes de resposta LogResponse para o líder.

Os outros nós só aceitariam pacotes LogRequest que incrementam o `commit_length` se apresentarem uma assinatura digital válida de uma maioria dos nós.

Demonstração das Medidas de Mitigação:

Não relacionado com a unidade curricular.

Conclusão

O que foi positivo:

- Excelente escolha do *Stateright*.
- Aprofundamento do conhecimento sobre o algoritmo de *Raft*.

O que melhorar:

- Executar nós *Raft* com as faltas implementadas ativadas dinamicamente.
- Integração mais direta com o *Stateright*:
 1. Permitindo a execução da sua UI, por exemplo.
 2. Cenários de teste definidos dinamicamente;



Stateright Explorer [API Docs](#) [Book](#) [Crate](#) [Source](#)

Status Run to completion

- **Model:** ActorModel<RaftActor<(), ()>>
- **States:** 343 827
- **Unique States:** 129 604
- **Max Depth:** 7
- **Progress:** [Deliver { src: Id(1), dst: Id(0), msg: ClientRequest(Write { key: 1, value: 42, msg_id: 1 }) }]

Properties ☐ Current state? ☐ Per-state?

Global

- ☒ **Example found:** Sometimes Election Liveness
- ☐ **Searching:** Sometimes Log Liveness
- ☐ **Searching:** Always Election Safety
- ☐ **Searching:** Always Log Safety

Path of Actions

1. Pre-init

Next Action Choices

- Init 0

Figura 1: Interface gráfica do *Stateright*